

# Лабораторные работы по дисциплине

## «JavaScript и веб-разработка»

---

Курс из 16 занятий (1 раз в неделю). Каждая лабораторная сдаётся коммитом в репозиторий `web-course-2026`, в папку `taskN/` (номер соответствует лабораторной, если не указано иное). В `README.md` каждой папки — 2–3 строки с описанием, что сделано и как запустить.

---

## Лабораторная работа №0. Организация репозитория

---

**Цель:** подготовить единое пространство, куда будут сдаваться все последующие работы.

**Что нужно сделать:**

1. Зарегистрироваться на GitHub (если аккаунта ещё нет).
2. Создать **публичный** репозиторий с именем `web-course-2026`.
3. Склонировать репозиторий локально (`git clone ...`).
4. В корне создать файл `README.md`, в котором указать:
  - ФИО;
  - номер группы;
  - список папок `task0 ... task16` (создать пустыми, с одним файлом `.gitkeep` в каждой, чтобы Git отслеживал пустые папки).
5. Добавить файл `.gitignore` в корень со строками:

```
node_modules/  
.DS_Store  
dist/  
.env
```

6. Сделать коммит с сообщением `init: repository structure` и запустить в `main`.
7. Включить GitHub Pages: Settings → Pages → Source: `main / root` (или `/docs`, если так удобнее). В `task0/` положить простой `index.html` с текстом «Курс JS — <ФИО>» и убедиться, что страница открывается по ссылке `https://<username>.github.io/web-course-2026/task0/`.

**Ограничения:** репозиторий должен быть публичным — иначе преподаватель не сможет проверить работу.

**Что сдавать:** ссылку на репозиторий и ссылку на опубликованную страницу GitHub Pages (одним сообщением/в ЛМС).

**Критерии проверки:** репозиторий существует, README заполнен, Pages работает, история коммитов не одним гигантским коммитом «всё сразу».

# Лабораторная работа №1. Семантическая вёрстка.

## Страница-резюме

---

**Цель:** научиться размечать страницу семантическими тегами HTML5 и располагать блоки с помощью Flexbox без использования float и абсолютного позиционирования.

**Что нужно сделать:** Сверстать одностраничное резюме/портфолио (можно про себя, можно про вымышленного персонажа). Страница обязана содержать следующие блоки:

1. **Шапка** ( `<header>` ) — имя, должность/специальность, короткий слоган.
2. **Фотография/аватар** — любое изображение (можно placeholder, например через `https://i.pravatar.cc/300` ).
3. **Блок «О себе»** ( `<section>` или `<article>` ) — 3–5 предложений биографии.
4. **Блок «Образование»** — список ( `<ul>/<ol>` ) с местом и годами обучения.
5. **Блок «Навыки»** — список в виде тегов/чипсов (например, каждый навык — `<span class="skill">` , оформленный как «пилюля» с фоном и скруглением).
6. **Блок «Контакты»** ( `<footer>` или отдельная `<section>` ) — минимум 3 интерактивные ссылки: `mailto:` , ссылка на соцсеть/GitHub, ссылка на телефон ( `tel:` ). Ссылки должны иметь ховер-эффект (смена цвета/подчёркивания через `:hover` ).
7. **Навигация** ( `<nav>` ) — если страница длинная, добавить якорные ссылки на блоки ( `<a href="#skills">` ).

### Технические требования (обязательные):

- Ровно один файл `index.html` и один файл `style.css` в папке `task1/` . Никакого JavaScript.
- HTML5-доктайп, `<meta charset="utf-8">` , `<meta name="viewport" content="width=device-width, initial-scale=1">` .
- Использовать **только семантические теги** для структуры: `<header>` , `<main>` , `<section>` , `<article>` , `<nav>` , `<footer>` . Использование «голого» `<div>` для структурных блоков — минус к оценке (внутри блоков `<div>` для вспомогательной вёрстки — можно).
- Вся раскладка блоков (расположение секций друг относительно друга, выравнивание элементов внутри блока) — **только через** `display: flex` . Использование `float` или `position: absolute/fixed` для основной раскладки — запрещено и обнуляет часть оценки за это задание. (Точечное использование `position: relative` для мелких декоративных элементов допустимо.)
- Обязательно продемонстрировать в CSS: `justify-content` , `align-items` , `flex-direction` , `flex-wrap` , `gap` .
- `box-sizing: border-box` должен быть выставлен глобально ( `* { box-sizing: border-box; }` ).
- Стили — во внешнем файле `style.css` , инлайн-стили ( `style="..."` ) и `<style>` внутри HTML — не использовать (кроме исключительных случаев, которые нужно уметь объяснить).

**Что НЕ надо делать:** не нужна адаптивность под мобильные устройства и медиазапросы — это следующая лабораторная. Не нужен JavaScript.

**Что сдавать:** папку `task1/` с `index.html`, `style.css`, изображениями (если локальные). Работа должна открываться через GitHub Pages по ссылке `.../task1/`.

**Критерии проверки:** валидная семантическая структура, отсутствие `float/absolute` в раскладке, использование Flexbox с как минимум 4 разными свойствами, отсутствие инлайн-стилей, страница открывается без ошибок в консоли.

---

## Лабораторная работа №2. Адаптивная вёрстка. Лендинг по макету

---

**Цель:** освоить CSS Grid, подход Mobile First и медиазапросы для адаптации под три класса устройств.

**Материалы:** макет в Figma выдаётся преподавателем (одностраничный лендинг — услуга/продукт/мероприятие на выбор).

**Что нужно сделать:**

1. Сверстать лендинг с адаптацией под три контрольные точки:
  - мобильный: до 767px;
  - планшет: 768px – 1199px;
  - десктоп: от 1200px.
2. Основная структурная сетка страницы (расположение крупных блоков: шапка, герой-секция, карточки услуг/товаров, футер и т.д.) — через **CSS Grid** (`display: grid`, `grid-template-columns`, `grid-template-rows`, `gap`, `grid-column/grid-row`).
3. Раскладка внутри отдельных компонентов (например, содержимое одной карточки, навигационное меню, кнопки с иконкой) — через **Flexbox**.
4. Меню должно визуально превращаться в «гамбургер» на мобильной ширине (иконку можно нарисовать через `<span>` + CSS или SVG). **JavaScript-логику клика реализовывать не нужно** — только то, как это должно выглядеть (пункт 4 из требований проверяется по CSS, не по функциональности).
5. Добавить форму обратной связи: минимум поля «Имя», «Email», «Сообщение» и кнопка «Отправить». Оформить `input`, `textarea`, `button` со стилями для обычного состояния, `:hover`, `:focus`. Отправка формы не реализуется (нет действия `action`, нет JS) — только вёрстка.
6. Добавить хотя бы один плавный переход (`transition`) — например, изменение цвета кнопки при наведении, или `transform: scale()` для карточки при `hover`.

**Технические требования (обязательные):**

- Подход **Mobile First**: базовые стили (без медиазапроса) пишутся для мобильного экрана, затем добавляются `@media (min-width: 768px) { ... }` и `@media (min-width: 1200px) { ... }`, расширяющие/переопределяющие стили. Медиазапросы вида `@media (max-width: ...)`, написанные «от десктопа к мобильному», не принимаются.
- Обязательно использование и Grid, и Flexbox в одном проекте (Grid — для крупной сетки страницы, Flexbox — для компонентов). Проект, целиком сверстанный только на Flexbox без единого `display: grid`, не засчитывается.
- Файлы: `task2/index.html`, `task2/style.css`. Изображения — либо ссылками, либо в подпапке `task2/img/`.
- Сходство с макетом Figma должно быть в разумных пределах: отступы, шрифты, цвета — близко к оригиналу (пиксель-в-пиксель не требуется, но пропорции и структура должны совпадать).
- Проверить вёрстку на реальном телефоне или через режим устройства в DevTools (Ctrl+Shift+M) — не должно быть горизонтального скролла, элементы не должны вылезать за экран, размеры кнопок/полей — не менее 44×44px (touch-friendly).

**Что сдавать:** ссылку на макет Figma, использованный вами (если преподаватель выдавал несколько на выбор — указать, какой), и папку `task2/`.

**Критерии проверки:** есть все три контрольные точки, стили написаны mobile-first, используются и Grid, и Flexbox по назначению, форма и меню оформлены, нет горизонтального скролла на мобильной ширине.

---

## Лабораторная работа №3. Основы JS и DOM: Todo-list

---

**Цель:** научиться работать с DOM-деревом, обрабатывать события и управлять состоянием интерфейса на чистом JavaScript.

**Что нужно сделать:** Реализовать интерактивный список задач (без сохранения данных между перезагрузками страницы — localStorage здесь использовать не нужно, это будет в лабораторной №9).

Функциональность:

1. Поле ввода (`<input type="text">`) и кнопка «Добавить» (или отправка по Enter — сделать оба варианта).
2. При добавлении задачи (текст не должен быть пустым — если поле пустое, задача не добавляется и/или показывается предупреждение) она появляется в списке под полем ввода.
3. У каждой задачи в списке должно быть:
  - чекбокс или кнопка «Выполнено» — по клику задача визуально помечается как выполненная (например, зачёркивание текста через CSS-класс `.completed { text-decoration: line-through; opacity: 0.6; }`);
  - кнопка «Удалить» — удаляет задачу из списка.

4. Счётчик над или под списком: «Осталось: N, Выполнено: M».
5. Фильтр из трёх кнопок/вкладок: «Все», «Активные», «Завершённые». При клике на фильтр список визуально показывает только соответствующие задачи (остальные скрываются, например через `display: none`, а не удаляются из данных).

### Технические требования (обязательные):

- Вся логика — в одном файле `script.js`, подключённом к `task3/index.html` через `<script src="script.js" defer></script>`.
- HTML-разметка отдельных задач **генерируется в JavaScript** (через `document.createElement` или сборку строки и `innerHTML`), не прописывается заранее вручную в HTML.
- Список задач должен храниться в JS в виде массива объектов, например:

```
// { id: 1, text: "Сделать лабу", completed: false }
```

Отрисовка списка на экране должна происходить на основе этого массива (функция `render()`, которая перерисовывает список при любом изменении данных).

- Использовать методы массивов `map` / `filter` / `forEach` там, где это уместно (например, `filter` — для отображения по текущему фильтру, `map` — при рендере).
- Каждой задаче должен присваиваться уникальный идентификатор (можно просто счётчик, увеличивающийся на 1, или `Date.now()`).
- Обработка событий — через `addEventListener` (не через атрибуты `onclick="..."` в HTML).
- В консоли браузера (F12) не должно быть ни одной ошибки при использовании приложения (добавление, удаление, отметка выполнения, переключение фильтров).

**Что сдавать:** `task3/index.html`, `task3/style.css`, `task3/script.js`.

**Критерии проверки:** все 5 пунктов функциональности работают, данные хранятся в массиве и рендерятся из него, нет `onclick` в HTML, нет ошибок в консоли.

---

## Лабораторная работа №4. Игра «Быки и коровы»

---

**Цель:** закрепить работу с состоянием приложения, массивами/строками и условной логикой без использования Canvas.

**Правила игры (если незнакомы):** компьютер загадывает число из 4 неповторяющихся цифр (от 0 до 9). Игрок вводит своё предположение — тоже 4 цифры. После каждой попытки показывается результат: количество «быков» (цифра угадана и стоит на своём месте) и «коров» (цифра есть в загаданном числе, но стоит на другом месте).

**Что нужно сделать:**

1. При загрузке страницы (или по кнопке «Новая игра») генерируется случайное 4-значное число с неповторяющимися цифрами.
2. Поле ввода для попытки игрока + кнопка «Проверить».

3. **Валидация ввода:** ровно 4 цифры, все цифры разные, только цифры (без букв/символов).  
При некорректном вводе — сообщение об ошибке, попытка не засчитывается.
4. После каждой попытки в списке (историю) добавляется строка вида: 1234 → 1 бык, 2 коровы.
5. Счётчик количества попыток.
6. Проверка на победу: если «быков» равно 4 — игра завершена, показывается сообщение «Победа! Угадано за N попыток» и поле ввода блокируется до нажатия «Новая игра».
7. Кнопка «Новая игра» — сбрасывает состояние (загаданное число, историю попыток, счётчик).

#### Технические требования (обязательные):

- Логика генерации числа, сравнения и подсчёта быков/коров — написать самостоятельно (без готовых прм-библиотек).
- Использовать функции: отдельная функция для генерации числа, отдельная — для валидации ввода, отдельная — для подсчёта быков и коров. Не писать всё одним большим обработчиком клика.
- Файлы: `task4/index.html`, `task4/style.css`, `task4/script.js` (Canvas в этой лабораторной не нужен, обычный DOM).
- История попыток должна рендериться из массива данных (как в лабораторной №3), а не добавляться напрямую в `innerHTML` без хранения состояния.

**Что сдавать:** папку `task4/`.

**Критерии проверки:** корректность подсчёта быков/коров на разных наборах входных данных (преподаватель проверит вручную несколькими комбинациями), валидация ввода работает, есть возможность сыграть заново без перезагрузки страницы.

---

## Лабораторная работа №5. Игра Simon (Симон говорит)

---

**Цель:** освоить работу с таймерами/задержками, звуком (по желанию) и последовательностями событий.

**Референс:** можно ориентироваться на [freesimon.org](https://freesimon.org), но верстка/цвета — свои, не копировать 1-в-1 чужой код.

**Правила игры:** на экране 4 цветные кнопки/сектора. Компьютер «показывает» последовательность (по очереди подсвечивает кнопки с паузой между ними). Игрок должен повторить последовательность в том же порядке, нажимая на кнопки. Если повторил верно — добавляется ещё один шаг к последовательности, и раунд повторяется заново (с начала, с уже более длинной последовательностью). Если ошибся — игра заканчивается, показывается достигнутый результат (длина последовательности).

**Что нужно сделать:**

1. Разметка: 4 кликабельных сектора/кнопки разного цвета, расположенных как единая фигура (круг из 4 частей или сетка 2×2) — оформление на Flexbox/Grid, как удобно.
2. Кнопка «Старт».
3. По нажатию «Старт»: последовательность из одного случайного элемента (0–3) генерируется и «проигрывается» — по очереди кнопки подсвечиваются (например, класс `.active` на время ~500мс) с паузой между ними.
4. После показа игрок кликает по кнопкам, повторяя последовательность. Клики должны блокироваться/игнорироваться, пока идёт показ последовательности компьютером (чтобы нельзя было кликать во время анимации).
5. Если игрок ввёл верную последовательность целиком — генерируется следующий раунд: к последовательности добавляется ещё один случайный элемент, и она снова показывается целиком с начала (по классике игры Simon).
6. Если игрок ошибся на любом шаге — игра завершается, выводится счёт («Вы дошли до уровня N»), кнопки блокируются до повторного нажатия «Старт».
7. Отображение текущего уровня/раунда на экране.

### Технические требования (обязательные):

- Для показа последовательности с задержками использовать `setTimeout` (лично реализовать очередь показа — например, рекурсивно или через `async/await` с `Promise` + `setTimeout` внутри). Использование `setInterval` без остановки — так, чтобы таймер продолжал тикать после ошибки/после уничтожения игры — не допускается: все таймеры должны корректно завершаться.
- Состояние игры (текущая последовательность, ввод игрока, флаг «идёт показ / ждём ввода») хранить в переменных/объекте, а не «угадывать» его через CSS-классы на странице.
- Файлы: `task5/index.html`, `task5/style.css`, `task5/script.js`.
- Звук — по желанию (плюс к оценке, не обязателен). Если добавляете — использовать короткие аудиофайлы или `AudioContext`, звук не должен блокировать логику игры.

**Что сдавать:** папку `task5/`.

**Критерии проверки:** последовательность корректно удлиняется на 1 каждый успешный раунд, во время показа клики не засчитываются, игра корректно завершается при ошибке без «зависших» таймеров (открыть консоль и проверить, что после нескольких игр подряд нет наложения старых и новых последовательностей).

---

## Лабораторная работа №6. Игра на Canvas («Змейка»)

---

**Цель:** освоить рисование на Canvas, игровой цикл через `requestAnimationFrame` и обработку клавиатуры.

**Что нужно сделать:** Реализовать классическую «Змейку»:

1. `<canvas>` фиксированного размера (например, 400×400 или 600×600, с сеткой из клеток одинакового размера, например 20×20 клеток по 20px).
2. Змейка изначально состоит из 3 сегментов, движется в одну сторону.
3. Управление стрелками клавиатуры ( `ArrowUp/Down/Left/Right` ). Змейка не может мгновенно развернуться на 180° (например, если движется вправо, нажатие «влево» не должно приводить к немедленному врезанию в саму себя — либо игнорировать такое нажатие, либо это должно приводить к проигрышу — выберите и придерживайтесь одного варианта, но не допускайте зависания).
4. На поле в случайной свободной клетке появляется «яблоко» (отрисованное как квадрат/круг другого цвета).
5. При «съедании» яблока змейка увеличивается на 1 сегмент, счёт растёт, на поле появляется новое яблоко в новом случайном месте (не на теле змейки).
6. Столкновение со стеной или с собственным хвостом → игра окончена.
7. Отображение текущего счёта — либо прямо на Canvas ( `ctx.fillText` ), либо HTML-элементом рядом с полем.
8. При окончании игры — надпись «Игра окончена» с итоговым счётом и кнопка «Начать заново» (можно и на канвасе, и как HTML-кнопка).

#### Технические требования (обязательные):

- Игровой цикл реализовать через `requestAnimationFrame` , **не через** `setInterval` .
- Скорость движения змейки должна быть **постоянной независимо от частоты кадров устройства** — то есть перемещение змейки на одну клетку должно происходить по расчёту прошедшего времени ( `deltaTime` ), а не на каждый вызов `requestAnimationFrame` (который может вызываться 60+ раз в секунду — так змейка будет двигаться слишком быстро).  
Пример подхода: хранить `lastMoveTime` , двигать змейку только если `currentTime - lastMoveTime >= moveInterval` (например, 150мс).
- Логика игры (состояние змейки — массив координат сегментов, направление, положение яблока, счёт, флаг `gameOver` ) должна храниться в переменных/объекте отдельно от отрисовки. То есть должны быть отдельные функции: `update()` (обновление состояния) и `draw()` (только отрисовка на основе текущего состояния) — не смешивать логику и рисование в одной функции без разделения.
- Файлы: `task6/index.html` , `task6/style.css` , `task6/script.js` .
- Код должен быть разбит на именованные функции ( `spawnApple()` , `checkCollision()` , `moveSnake()` и т.д.), а не быть одним файлом из 200 строк в одном обработчике.

**Что сдавать:** папку `task6/` .

**Критерии проверки:** змейка двигается с одинаковой скоростью на разных устройствах/мониторах (проверяется визуально), корректно обрабатываются столкновения, игра перезапускается без перезагрузки страницы, код разбит на функции.

*Альтернатива по согласованию с преподавателем: «Тетрис» или «Крестики-нолики» с простым компьютерным соперником (случайный ход или блокировка очевидной победы) — если*

хотите более сложную задачу вместо змейки, требования по структуре кода ( `update / draw` , разбиение на функции, `requestAnimationFrame` ) сохраняются.

---

## Лабораторная работа №7. Асинхронность: лента постов с бесконечным скроллом

---

**Цель:** освоить `fetch` , `Promise / async-await` , обработку ошибок сети и фильтрацию данных с сервера.

**API:** [jsonplaceholder.typicode.com](https://jsonplaceholder.typicode.com)

**Что нужно сделать:**

1. При загрузке страницы подгружается первая «порция» постов (например, 10 штук) через `fetch K https://jsonplaceholder.typicode.com/posts` .
2. Каждый пост отображается карточкой: заголовок ( `title` ), текст ( `body` ), id автора ( `userId` ).
3. **Бесконечный скролл:** при прокрутке страницы до конца текущего списка (например, когда пользователь долистал до последних 200px страницы) автоматически подгружается следующая порция постов и добавляется в конец списка (для реализации отслеживания скролла можно использовать событие `scroll` на `window` с проверкой позиции, либо `IntersectionObserver` — на выбор).
4. Пока постов больше нет (всего в API 100 постов) — подгрузка останавливается, показывается сообщение «Больше постов нет».
5. Поле поиска ( `<input>` ) с фильтром **по подстроке**: при вводе текста список фильтруется так, чтобы отображались только посты, у которых `title` содержит введённую подстроку (без учёта регистра). Фильтрация происходит **по уже загруженным данным на клиенте** (не отправляется новый запрос на каждый символ ввода).
6. Пока идёт загрузка очередной порции — показывать индикатор загрузки (текст «Загрузка...» или простой CSS-спиннер).
7. Обработка ошибки сети: если `fetch` завершился с ошибкой или `response.ok === false` , показать пользователю понятное сообщение об ошибке (не молчаливый сбой в консоли).

**Технические требования (обязательные):**

- Использовать `async/await` (не цепочки `.then()` — это допустимо в отдельных местах, но основной код должен быть на `async/await` ).
- Каждый `fetch` -запрос обернут в `try/catch` .
- Все загруженные посты хранятся в общем массиве в JS; отрисовка (и фильтрация) идёт на основе этого массива.
- Дебаунс для поля поиска — задержка ~300мс перед применением фильтра после того, как пользователь прекратил печатать (реализовать самостоятельно через `setTimeout / clearTimeout` , без библиотек).
- Файлы: `task7/index.html` , `task7/style.css` , `task7/script.js` .

Что сдавать: папку `task7/`.

**Критерии проверки:** подгрузка при скролле работает, останов после исчерпания данных, фильтр по подстроке работает без лишних запросов к серверу, есть обработка ошибок и индикатор загрузки.

---

## Лабораторная работа №8–9. Zen-Money — полноценное full-stack приложение (2 занятия, «два в одном»)

---

**Цель:** написать **собственный** сервер на Node.js/Express с реальной базой данных (SQLite) и полноценный фронтенд к нему. Это объёмная лабораторная, объединяющая backend и frontend — оценивается как единая работа с двумя равнозначными частями.

Почему не `json-server`: `json-server` подделывает REST API за вас — вы отправляете запрос и получаете ответ, не понимая, что происходит внутри. Цель этой лабораторной — самим написать то, что раньше делал `json-server` за вас, и увидеть API «изнутри».

### Часть 1 (занятие 8): backend — Express + SQLite

---

**Подготовка проекта:**

1. Создать папку `task8-9/server/`, внутри — `npm init -y`.
2. Установить зависимости: `npm install express better-sqlite3 cors`.
  - `express` — сам сервер;
  - `better-sqlite3` — работа с SQLite синхронно и просто (без промисов, что упрощает код для первого знакомства с БД);
  - `cors` — чтобы фронтенд (запущенный на другом порту/через Live Server) мог обращаться к серверу без блокировки браузером.

**Структура базы данных (создать при старте сервера, если таблиц ещё нет):**

```
CREATE TABLE IF NOT EXISTS categories (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  type TEXT NOT NULL CHECK(type IN ('income', 'expense'))  
);  
  
CREATE TABLE IF NOT EXISTS transactions (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  amount REAL NOT NULL,  
  category_id INTEGER NOT NULL,  
  date TEXT NOT NULL,  
  comment TEXT,  
  FOREIGN KEY (category_id) REFERENCES categories(id)  
);
```

При первом запуске сервера, если таблица `categories` пустая, добавить программно 2-3 стартовые категории (например, «Еда» — `expense`, «Зарплата» — `income`), чтобы фронтенд сразу было с чем тестировать.

**Что нужно реализовать на сервере ( `server.js` или разбито на файлы `routes/` , `db.js` — на ваш выбор структуры, но не всё в одном файле длиннее 150 строк):**

| Метод  | Роут                           | Что делает                                                                                                                                                                                                                                |
|--------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET    | <code>/categories</code>       | вернуть все категории                                                                                                                                                                                                                     |
| POST   | <code>/categories</code>       | создать категорию ( <code>name</code> , <code>type</code> в теле запроса)                                                                                                                                                                 |
| GET    | <code>/transactions</code>     | вернуть все операции; поддержать query-параметры <code>?category_id=</code> , <code>?type=</code> , <code>?from=&amp;to=</code> (диапазон дат) — фильтрация должна происходить <b>в SQL-запросе</b> ( <code>WHERE</code> ), не на клиенте |
| POST   | <code>/transactions</code>     | создать операцию (проверить, что <code>amount</code> — число больше 0, <code>category_id</code> существует — иначе вернуть статус 400 с телом { <code>"error": "..."</code> } )                                                           |
| PUT    | <code>/transactions/:id</code> | обновить операцию по id                                                                                                                                                                                                                   |
| DELETE | <code>/transactions/:id</code> | удалить операцию по id                                                                                                                                                                                                                    |

**Технические требования (обязательные):**

- Использовать middleware `express.json()` для разбора тела запроса и `cors()` — подключить глобально.
- Каждый роут обернут в `try/catch` , при ошибке — ответ со статусом 500 и понятным телом { `"error": "..."` } , не «упавший» без ответа сервер.
- На несуществующий id (например, `DELETE /transactions/999` , которого нет) — ответ 404 , а не 200 / 500 .
- Валидация входных данных на **сервере обязательна**, даже если фронтенд тоже валидирует — сервер не должен доверять клиенту.
- Сервер запускается командой `node server.js` (или `npm start` , прописанной в `package.json` ) на порту 3001 .
- В `server/README.md` — команды установки ( `npm install` ) и запуска.

**Как проверить сервер без фронтенда (обязательный шаг перед переходом к части 2):**  
протестировать все 6 роутов через Postman, Thunder Client (расширение VS Code) или просто `curl` — приложить в README скриншот или лог хотя бы одного успешного запроса каждого типа (GET/POST/PUT/DELETE).

**Часть 2 (занятие 9): frontend + фильтры + деплой связки**

---

**Что нужно сделать во фронтенде** (папка `task8-9/client/` , обращается к `http://localhost:3001` ):

- Форма добавления операции: сумма, тип, категория (выпадающий список из `GET /categories` ), дата, комментарий → `POST /transactions` .
- Список операций через `GET /transactions` , карточками/таблицей: доходы — зелёным, расходы — красным.
- Удаление ( `DELETE` ) и редактирование ( `PUT` ) операции — как в обычном CRUD.

- Итоговая сводка «Доходы / Расходы / Баланс» — считать на фронтенде на основе загруженного списка.
- Фильтры по категории, типу и диапазону дат** — теперь это не фильтрация массива на клиенте, а **параметры GET-запроса** к серверу ( `fetch('/transactions?type=expense&from=2026-01-01')` ), которые сервер обрабатывает в SQL. Это ключевое отличие от версии на `json-server` — фильтрация переезжает на бэкэнд.
- Возможность создать свою категорию → `POST /categories`.
- Сохранение в `localStorage` — только пользовательские настройки интерфейса (последний фильтр, тема). Сами данные — исключительно на сервере в SQLite.

### Технические требования:

- Код фронтенда разбит на ES-модули: `api.js` (все функции запросов к серверу), `ui.js` (отрисовка), `main.js` (точка входа).
- Все запросы — `fetch + async/await + try/catch`; при ошибке сервера (400/404/500) — показать пользователю сообщение, а не тихо проглотить ошибку.
- Простая клиентская валидация формы (по UX — чтобы не ждать ответа сервера на явно пустое поле), но она не заменяет серверную из части 1.

**Что сдавать:** папку `task8-9/` с подпапками `server/` и `client/`, файл базы `database.sqlite` можно не коммитить (добавить в `.gitignore`), но приложить пару строк в README, как база создаётся автоматически при первом запуске. Обязательно — инструкция запуска обеих частей (сначала сервер, потом клиент).

**Критерии проверки (единые для обеих частей):** сервер запускается и отвечает на все 6 роутов корректными статусами, валидация есть и на сервере, и на клиенте, фильтрация происходит через SQL-запрос (не `.filter()` на клиенте — это проверяется по коду), полный CRUD работает end-to-end через интерфейс, нет ошибок в консоли браузера и в терминале сервера при обычном сценарии использования.

---

## Лабораторная работа №10–11. Todo-list с фильтрами и сортировкой на React (2 занятия)

---

**Цель:** переписать логику, аналогичную лабораторной №3, на React, освоив компоненты, `props`, `useState`, `useEffect`.

**Подготовка:** создать проект через `npm create vite@latest todo-react -- --template react` (Vite рекомендуется как более быстрый инструмент, но `create-react-app` тоже допустим).

### Часть 1 (занятие 10): компоненты и состояние

---

**Что нужно сделать:**

1. Разбить интерфейс на компоненты: `App`, `TodoForm` (поле ввода + кнопка добавления), `TodoList` (список), `TodoItem` (одна задача), `Filters` (кнопки фильтра), `Stats` (счётчики).
2. Состояние списка задач хранится в `App` через `useState` и передаётся дочерним компонентам через `props`. Компонент `TodoItem` не должен хранить собственную копию текста задачи в своём состоянии — только получать через `props` и вызывать переданные из `App` функции (`onToggle`, `onDelete`) по клику.
3. Реализовать те же функции, что и в лабораторной №3: добавление, отметка выполнения, удаление, фильтр «Все / Активные / Завершённые».
4. Каждый элемент списка при рендере (`.map()`) должен иметь корректный `key` (использовать `id` задачи, **не индекс массива**, если только вы не готовы объяснить, почему индекс тут допустим).

### Технические требования:

- Только функциональные компоненты и хуки (никаких классовых компонентов).
- `props` должны иметь типизацию хотя бы в виде комментариев JSDoc или деструктуризации с понятными именами — жёсткого требования TypeScript нет, но код должен быть читаем.

## Часть 2 (занятие 11): сортировка, `useEffect` и работа с API (по желанию — расширение)

---

### Что нужно добавить:

1. Сортировка списка: по алфавиту ( $A \rightarrow Я / A \rightarrow Z$ ), по дате добавления (сначала новые / сначала старые). Реализовать через выпадающий список или кнопки, сортировка не должна мутировать исходный массив напрямую (использовать `[...items].sort(...)`, а не `items.sort(...)`).
2. Использовать `useEffect` минимум один раз осмысленно — например, для сохранения списка задач в `localStorage` при каждом изменении и восстановления при загрузке приложения (`useEffect` с пустым массивом зависимостей — для загрузки при монтировании; `useEffect`, зависящий от списка задач, — для сохранения при каждом изменении).
3. (Опционально, плюс к оценке) Подключить тот же `json-server` из zen-money-проекта (или отдельный `db.json` с задачами) и хранить задачи на сервере вместо/вместе с `localStorage`, используя `fetch` внутри `useEffect`.

**Что сдавать:** папку `task10-11/` (или отдельный репозиторий/поддиректорию с `package.json` — проговорить с преподавателем формат, если весь проект не помещается логично в `web-course-2026`, можно дать ссылку на отдельный репозиторий в README).

**Критерии проверки:** компоненты корректно разделены и обмениваются данными через `props`, нет классовых компонентов, `key` на месте, сортировка не ломает исходные данные, `useEffect` используется по назначению (не как замена обработчику клика).

---

# Лабораторная работа №12–13. Zen-Money на Next.js — со своим API (2 занятия)

---

**Цель:** перенести проект «Zen-Money» на Next.js, освоить файловый роутинг, а бэкенд из лабораторной №8–9 (Express + SQLite) перенести внутрь Next.js в виде **API routes** — увидеть, что серверная логика, написанная руками в Express, и API routes Next.js устроены по одному и тому же принципу.

## Часть 1 (занятие 12): каркас и перенос интерфейса

---

### Что нужно сделать:

1. Создать проект: `npx create-next-app@latest zen-money-next` (App Router, JavaScript — если TypeScript не проходили).
2. Перенести интерфейс zen-money на React-компоненты Next.js: минимум 2 роута — `app/page.js` (список операций + сводка), `app/add/page.js` (форма добавления). Навигация — через `next/link`, не `<a href="...">`.
3. Перенести бэкенд-логику из части 8–9 в `app/api/`: те же 6 сценариев (получить категории, создать категорию, получить операции с фильтрами, создать/обновить/удалить операцию) реализовать как **Route Handlers** Next.js:
  - `app/api/categories/route.js` — GET И POST;
  - `app/api/transactions/route.js` — GET (с чтением `searchParams` из URL для фильтров) И POST;
  - `app/api/transactions/[id]/route.js` — PUT И DELETE.
4. Подключение к базе — переиспользовать ту же библиотеку `better-sqlite3` и, по возможности, тот же файл базы данных (или пересоздать структуру таблиц идентично части 8–9) — вынести подключение в отдельный файл `lib/db.js`, который импортируется во все route handlers (не открывать соединение с базой заново в каждом файле).
5. Компоненты, которые делают запросы к `/api/...` (внутренние, относительные пути — не `http://localhost:3001`, теперь всё в одном Next.js-проекте), должны быть клиентскими ("use client" в начале файла), так как используют `useState` / `useEffect` / события.

## Часть 2 (занятие 13): динамический роут, объяснение архитектуры, деплой

---

### Что нужно сделать:

1. Реализовать страницу подробного просмотра операции: `app/transaction/[id]/page.js` — получает `id` из параметров URL, запрашивает `GET /api/transactions/:id` (добавить этот метод, если не было в части 1), отображает детали.
2. Разобраться и кратко (2–3 предложения в README) объяснить своими словами разницу между CSR, SSR и SSG применительно к своему проекту — и дополнительно: **чем API routes в Next.js похожи и чем отличаются от Express-сервера**, который вы писали в

- лабораторной №8–9 (оба принимают HTTP-запрос и возвращают ответ; но в Next.js это не отдельный процесс/порт, а часть того же приложения, задеплоенного как единое целое).
3. Задеплоить проект на Vercel ( `vercel.com` , привязка к GitHub-репозиторию). Так как теперь **вся серверная логика — часть самого Next.js-приложения** (а не отдельный локальный Express-сервер), задеплоенная версия на Vercel должна работать полноценно, с реальными данными — в отличие от предыдущей версии на json-server. Если используете SQLite-файл — учесть, что на Vercel файловая система при каждом деплое/обращении может пересоздаваться (serverless-окружение), и указать в README, наблюдали ли вы эту особенность на практике (это нормальный повод для короткого вывода в отчёте, а не повод считать лабораторную не выполненной).

**Что сдавать:** ссылку на репозиторий с Next.js-проектом (включая `app/api/` ), ссылку на деплой на Vercel, README с объяснениями по пункту 2.

**Критерии проверки:** роутинг реализован через файловую структуру `app/` , API routes корректно повторяют функциональность Express-сервера из №8–9 (все 6 сценариев работают), `next/link` используется вместо обычных ссылок, динамический роут работает, есть внятное (своими словами) объяснение разницы CSR/SSR/SSG и сходства API routes с Express, деплой на Vercel выполнен и по возможности работает с реальными данными.

---

## Лабораторная работа №14. Тестирование приложения

---

**Цель:** познакомиться с юнит-тестированием и тестированием React-компонентов.

**Что нужно сделать:**

1. Установить `Jest` (для проекта без React — например, для логики из лабораторной №4 «Быки и коровы») или `Vitest + React Testing Library` (для React-проекта — Todo-list из лабораторной №10-11).
2. **Для логики без UI** (рекомендуется взять функцию подсчёта быков/коров из лабораторной №4 или функцию сортировки/фильтрации из Todo-list): написать минимум 5 юнит-тестов, покрывающих:
  - обычный случай («средний» ввод даёт ожидаемый результат);
  - краевой случай (например, полное совпадение — 4 быка, 0 коров);
  - случай с отсутствием совпадений;
  - как минимум один тест на некорректные входные данные (если функция должна на них реагировать определённым образом).
3. **Для React-компонента** (Todo-list): написать минимум 3 теста с использованием `React Testing Library` , например:
  - компонент `TodoForm` рендерится и содержит поле ввода и кнопку;
  - при вводе текста и клике на «Добавить» вызывается переданная через props функция-обработчик ( `jest.fn()` / `vi.fn()` ) и проверка, что она была вызвана с ожидаемым аргументом);

- компонент `TodoItem` при клике на кнопку «Удалить» вызывает `onDelete` с правильным `id`.

4. Все тесты должны запускаться командой `npm test` (или `npx vitest run`) и **проходить** (зелёные) на момент сдачи.

### Технические требования:

- Тестовые файлы называть по конвенции `*.test.js` (или `*.spec.js`), располагать рядом с тестируемым файлом или в папке `__tests__` / .
- В README указать, какой командой запускать тесты и какой процент функциональности покрыт (можно приблизительно, без строгого требования по coverage-инструментам, но с обоснованием, какие сценарии протестированы, а какие сознательно оставлены без теста и почему).

**Что сдавать:** проект (тот же репозиторий с Todo-list или с «Быками и коровами», к которому добавлены тесты), скриншот успешного прохождения тестов в терминале (в README или отдельным файлом `tests-result.png`).

**Критерии проверки:** тесты действительно проверяют логику (не «заглушки» вида `expect(true).toBe(true)`), тесты проходят, минимальное количество тестов соблюдено, тесты запускаются без ручной правки кода перед проверкой.

---

## Лабораторная работа №15–16. Введение в Angular + финальный проект (2 занятия)

---

**Цель:** познакомиться с архитектурой Angular (модули, компоненты, DI, RxJS-подход) в сравнении с React, закрепить материал курса на защите итогового проекта.

### Часть 1 (занятие 15): мини-проект на Angular

---

#### Что нужно сделать:

1. Установить Angular CLI (`npm install -g @angular/cli`), создать проект `ng new todo-angular` (routing — по желанию, стили — CSS).
2. Переписать **одну** из ранее сделанных работ на Angular — на выбор: Todo-list (лабораторная №3/10) или «Быки и коровы» (лабораторная №4).
3. Реализовать минимум:
  - один компонент со свойствами и методами (`@Component`);
  - `[(ngModel)]` (двустороннее связывание) для поля ввода — потребуется подключить `FormsModule`;
  - `*ngFor` для рендера списка (задач или истории попыток);
  - `*ngIf` для условного отображения (например, сообщение «Список пуст» или «Игра окончена»);

- как минимум один **сервис** (`@Injectable`), вынесенный из компонента через `Dependency Injection`, отвечающий за хранение/обработку данных (аналог того, что в `React` вы делали через `useState` в родительском компоненте, — здесь это выносится в отдельный класс-сервис, который внедряется в компонент через конструктор).
4. В README (2–4 предложения, своими словами) сравнить подход `Angular` и `React` в вашем конкретном проекте: что показалось привычнее/непривычнее, чем `[(ngModel)]` /сервис с `DI` отличается от `useState / props` в `React`.

## Часть 2 (занятие 16): защита итогового проекта

---

### Что нужно сделать:

1. Подготовить короткую презентацию/демонстрацию (5–7 минут) одного проекта на выбор из выполненных за курс (рекомендуется — `Zen-Money` на `Next.js`, как самый комплексный, но допустим любой обоснованный выбор).
2. В демонстрации показать: работающее приложение (локально или задеплоенное), рассказать о структуре кода (компоненты/модули), ответить на 2–3 вопроса преподавателя по коду (например: «почему здесь используется `useEffect`», «зачем вынесен этот код в отдельный модуль», «как обрабатывается ошибка, если сервер недоступен»).
3. Сдать финальный чек-лист по всему репозиторию `web-course-2026`: все папки `task0` — `task14` (и `Angular/Next.js`-проекты — ссылками, если они вынесены в отдельные репозитории) должны быть на месте и рабочими.

**Что сдавать:** ссылку на итоговый проект для защиты, время для демонстрации (согласовать заранее), финальный репозиторий.

**Критерии проверки:** проект на `Angular` использует `*ngFor`, `*ngIf`, `[(ngModel)]` и минимум один сервис с `DI` — без этого лабораторная не засчитывается; на защите студент может объяснить собственный код (это главный критерий — не сдать чужой проект, не понимая, что в нём происходит).

---

## Общие правила для всех лабораторных работ

---

- Каждая лабораторная сдаётся **отдельными коммитами** (минимум 2–3 коммита с осмысленными сообщениями на каждую лабораторную — не один коммит «сделал всё»).
- Дедлайн — начало следующего занятия, если не оговорено иное. Лабораторная, сданная после начала следующего занятия, но до конца текущей недели, принимается с пометкой «сдано с опозданием» (влияет на итоговую оценку по накопительной системе — уточняется преподавателем отдельно).
- Списанный без понимания код (в том числе сгенерированный нейросетью без изменений и понимания) при неспособности объяснить его на устном опросе — не засчитывается. Использовать ИИ-инструменты как помощника — можно и не запрещается, но студент должен уметь объяснить каждую строчку своего кода.

- Все работы должны открываться и проверяться без дополнительной настройки окружения преподавателем, кроме случаев, явно описанных в задании (например, `npm install + npm run dev` для React/Next.js/Angular-проектов — это нормально и ожидаемо, инструкция по запуску должна быть в README).